

LAPORAN PRAKTIKUM 2
SISTEM OPERASI: PROSES



Disusun Oleh =
M. Ayatullah Mutawakki F (2410651011)

Dosen Pengampu =
Triawan Adi Cahyanto M.Kom

PROGRAM STUDY TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2025

PENDAHULUAN

A. TUJUAN

Memahami konsep proses dan siklus hidupnya — Pembaca diharapkan mampu menjelaskan pengertian proses dalam sistem operasi, tahapan siklus hidup proses (new, ready, running, waiting, terminated), serta memahami bagaimana sistem operasi melakukan manajemen terhadap berbagai status proses tersebut.

Mampu membuat dan mengelola proses menggunakan system call — Pembaca dapat menerapkan pemanggilan sistem seperti fork(), exec(), dan wait() untuk membuat, menjalankan, dan mengatur eksekusi proses secara terprogram menggunakan bahasa C di lingkungan sistem operasi Linux.

Memahami hubungan parent-child process — Mahasiswa mampu mengidentifikasi dan menjelaskan interaksi antara proses induk dan proses anak, termasuk bagaimana mereka berbagi sumber daya, serta cara sistem operasi mengatur hierarki proses.

Menganalisis Process ID (PID) dan Process Control Block (PCB) — Mahasiswa dapat menjelaskan fungsi dan peran PID sebagai identitas unik proses, serta memahami isi dan struktur PCB yang menyimpan informasi penting terkait status, konteks CPU, alokasi memori, dan informasi manajemen proses lainnya.

B. DASAR TEORI TOPIK 1 (Concept dan Opetation)

Proses adalah program yang sedang dijalankan (dieksekusi) oleh sistem operasi. Setiap proses memiliki:

- PID (Process ID) yang unik.
- PPID (Parent Process ID) yang menunjuk proses induknya.

Sistem operasi mengatur eksekusi proses menggunakan PCB (Process Control Block), yang menyimpan informasi seperti:

- Status proses,
- Register CPU,
- Informasi memori,
- Informasi manajemen file.

Pembuatan proses baru dilakukan dengan system call fork(). Setelah fork(), terdapat dua proses identik: parent dan child, yang bisa berinteraksi dengan wait() atau exec()).

C. DASAR TEORI TOPIK 2 (Interprocess Communication (IPC))

Interprocess Communication (IPC) adalah mekanisme yang digunakan sistem operasi agar beberapa proses dapat berkomunikasi dan bertukar data. IPC diperlukan karena setiap proses memiliki ruang memori terpisah. Beberapa metode umum IPC meliputi:

- Pipes: komunikasi satu arah antara parent dan child.
- FIFO (Named Pipe): komunikasi dua arah antar proses yang tidak memiliki hubungan langsung.
- Shared Memory: proses saling berbagi area memori yang sama, kecepatan tinggi.
- Message Queue / Socket: untuk komunikasi antar proses yang lebih kompleks.

D. PERCOBAAN DAN HASIL

TOPIK 1 CONCEPT DAN OPERATION

1. PERCOBAAN 1.1 (membuat proses sederhana)

Kode: process_basic.c

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
int main() {
    pid_t pid;

    printf("Sebelum fork()\n");
    printf("PID: %d\n\n", getpid());

    pid = fork();

    if (pid < 0) {
        fprintf(stderr, "Fork gagal!\n");
        return 1;
    }
    else if (pid == 0) {
        // Child process
        printf("CHILD PROCESS\n");
```

```

        printf("PID saya: %d\n", getpid());
        printf("PID parent saya: %d\n", getppid());
    }
    else {
        // Parent process
        printf("PARENT PROCESS\n");
        printf("PID saya: %d\n", getpid());
        printf("PID child saya: %d\n", pid);
    }

    printf("Proses %d selesai\n\n", getpid());

    return 0;
}

```

Compile dan jalankan:

```
gcc process_basic.c -o process_basic
```

```
./process_basic
```

```

aqil@DESKTOP-RGGJ3FG:~$ gcc process_basic.c -o process_basic
./process_basic
Sebelum fork()
PID: 639

PARENT PROCESS
PID saya: 639
PID child saya: 640
Proses 639 selesai

CHILD PROCESS
PID saya: 640
PID parent saya: 639
Proses 640 selesai

```

Analisis: Menunjukkan bahwa setiap program (proses) memiliki ID unik yang diberikan oleh kernel.

2. PERCOBAAN 1.2 (Fork: Parent dan Child Process)

Kode: fork_demo.c

```

#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid;
    int status;

    pid = fork();

    if (pid == 0) {
        // Child process
        printf("Child: PID saya %d, akan tidur 3 detik\n", getpid());
        sleep(3);
        printf("Child: Selesai!\n");
        return 42; // Exit code
    }
    else {
        // Parent process
        printf("Parent: Menunggu child (PID: %d) selesai...\n", pid);
        wait(&status);
        if (WIFEXITED(status)) {
            printf("Parent: Child selesai dengan exit code: %d\n",
                WEXITSTATUS(status));
        }
    }
    return 0;
}

```

Compaile dan jalankan:

```
gcc fork_demo.c -o fork_demo
```

```
./fork_demo
```

Output:

```
aqil@DESKTOP-RGGJ3FG:~$ gcc fork_demo.c -o fork_demo
aqil@DESKTOP-RGGJ3FG:~$ ./fork_demo
Sebelum fork()
PID: 727

PARENT PROCESS
PID saya: 727
PID child saya: 728
Proses 727 selesai

CHILD PROCESS
PID saya: 728
PID parent saya: 310
Proses 728 selesai
```

Analisis:

Setelah pemanggilan fork(), terbentuk dua proses berbeda:

- Parent process memiliki PID sendiri dan mengetahui PID anak.
- Child process memiliki PID baru dan mengenali parent-nya.

Kedua proses berjalan secara independen, menunjukkan konsep concurrent execution.

3. Percobaan 1.3 Process Termination dengan wait()

Kode: process_wait.c

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid;
    int status;

    pid = fork();
```

```

    if (pid == 0) {
        printf("Child: PID saya %d, akan tidur 3 detik\n", getpid());
        sleep(3);
        printf("Child: Selesai!\n");
        return 42;
    } else {
        printf("Parent: Menunggu child (PID: %d) selesai...\n", pid);
        wait(&status);
        if (WIFEXITED(status)) {
            printf("Parent: Child selesai dengan exit code: %d\n",
WEXITSTATUS(status));
        }
    }
    return 0;
}

```

Compile: gcc proses_wait.c -o proses_wait

./proses_wait

Hasilnya:

```

Parent: Menunggu child (PID: 762) selesai...
Child: PID saya 762, akan tidur 3 detik
Child: Selesai!
Parent: Child selesai dengan exit code: 42

```

Analisis:

Parent menunggu child selesai menggunakan wait(), memastikan urutan eksekusi yang benar.

Setelah child selesai, parent menerima exit code 42, menandakan komunikasi berhasil antara kedua proses.

4. Percobaan 1.4 Multiple Child Processes

Kode: multiple_fork.c

```

#include <stdio.h>
#include <unistd.h>

```

```

#include <sys/wait.h>

int main() {
    int i;
    pid_t pid;
    int num_children = 3;

    printf("Parent PID: %d\n\n", getpid());

    for (i = 0; i < num_children; i++) {
        pid = fork();
        if (pid == 0) {
            printf("Child %d: PID = %d, Parent PID = %d\n", i+1, getpid(),
getppid());
            sleep(i + 1);
            printf("Child %d selesai\n", i+1);
            return i;
        }
    }

    for (i = 0; i < num_children; i++) {
        int status;
        pid_t child_pid = wait(&status);
        printf("Parent: Child dengan PID %d selesai\n", child_pid);
    }

    printf("\nParent: Semua child selesai\n");
    return 0;
}

```

Compile:

Gcc multiple_fork.c -o multiple_fork

./multiple_fork

Hasil:

```
Parent PID: 795
Child 1: PID = 796, Parent PID = 795
Child 2: PID = 797, Parent PID = 795
Child 3: PID = 798, Parent PID = 795
Child 1 selesai
Parent: Child dengan PID 796 selesai
Child 2 selesai
Parent: Child dengan PID 797 selesai
Child 3 selesai
Parent: Child dengan PID 798 selesai
Parent: Semua child selesai
```

Analisis:

Program membuat tiga child process secara berurutan.

Parent menunggu semua child selesai satu per satu menggunakan wait().

Hasil menunjukkan proses paralel sederhana, di mana tiap child memiliki PID berbeda.

E. PERCOBAAN DAN HASIL

TOPIK 2 IPC

1. Topik 2.1 Pipes (Komunikasi Satu Arah)

Kode: pipe_demo.c

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>

int main() {
    int pipefd[2]; // pipefd[0] = read, pipefd[1] = write
    pid_t pid;
    char write_msg[] = "Hello dari Parent!";
    char read_msg[100];

    if (pipe(pipefd) == -1) {
        perror("Pipe gagal");
        return 1;
    }
}
```

```

    pid = fork();

    if (pid == 0) {
        close(pipefd[1]); // Child hanya baca
        read(pipefd[0], read_msg, sizeof(read_msg));
        printf("Child menerima: %s\n", read_msg);
        close(pipefd[0]);
    } else {
        close(pipefd[0]); // Parent hanya tulis
        printf("Parent mengirim: %s\n", write_msg);
        write(pipefd[1], write_msg, strlen(write_msg) + 1);
        close(pipefd[1]);
        wait(NULL);
    }

    return 0;
}

```

Compile: gcc pipe_demo.c -o pipe_demo

./pipe_demo

Hasil:

```

Parent mengirim: Hello dari Parent!
Child menerima: Hello dari Parent!

```

Analisis:

Pipe menyediakan saluran komunikasi satu arah antar proses.

Parent mengirim data melalui write-end (pipefd[1]), sedangkan child membaca dari read-end (pipefd[0]).

Metode ini cepat dan sederhana, namun hanya bisa digunakan antara proses yang memiliki hubungan parent-child.

2. Percobaan 2.2 – Named Pipes (FIFO, Two-Way Communication)

Kode: fifo_writer.c

```
#include <stdio.h>
```

```

#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>
#define FIFO_FILE "/tmp/my_fifo"

int main() {
    int fd;
    char buffer[100];

    mkfifo(FIFO_FILE, 0666);

    printf("Writer: Menunggu reader...\n");
    fd = open(FIFO_FILE, O_WRONLY);

    printf("Masukkan pesan: ");
    fgets(buffer, sizeof(buffer), stdin);

    write(fd, buffer, strlen(buffer) + 1);
    printf("Writer: Pesan terkirim\n");

    close(fd);
    return 0;
}

```

Kode: fifo_reader.c

```

#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>
#define FIFO_FILE "/tmp/my_fifo"

```

```

int main() {
    int fd;
    char buffer[100];

    printf("Reader: Membuka FIFO...\n");
    fd = open(FIFO_FILE, O_RDONLY);

    read(fd, buffer, sizeof(buffer));
    printf("Reader: Menerima pesan: %s\n", buffer);

    close(fd);
    unlink(FIFO_FILE); // Hapus FIFO
    return 0;
}

```

Langkah Eksekusi:

1. Buka dua terminal.
2. Terminal 1 → ./fifo_reader
3. Terminal 2 → ./fifo_writer

Hasil :

```

./fifo_reader
Writer: Menunggu reader...
Masukkan pesan: hai
Writer: Pesan terkirim

```

```

./fifo_writer
Reader: Membuka FIFO...
hai
Reader: Menerima pesan: hai

```

Analisis:

FIFO memungkinkan komunikasi antar proses yang tidak memiliki hubungan langsung.

Berbeda dari pipe biasa, FIFO memiliki nama permanen di sistem file (/tmp/my_fifo).

Metode ini cocok untuk komunikasi antar program yang berjalan di waktu berbeda.

3. Percobaan 2.3 Shared Memory

Kode: shm_writer.c

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <string.h>
#define SHM_SIZE 1024

int main() {
    key_t key;
    int shmid;
    char *shared_memory;

    key = ftok("shmfile", 65);
    shmid = shmget(key, SHM_SIZE, 0666 | IPC_CREAT);
    shared_memory = (char*) shmat(shmid, NULL, 0);

    printf("Writer: Masukkan data: ");
    fgets(shared_memory, SHM_SIZE, stdin);

    printf("Writer: Data tersimpan di shared memory.\n");

    shmdt(shared_memory);
```

```
    return 0;
}
```

Kode: shm_reader.c

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#define SHM_SIZE 1024

int main() {
    key_t key;
    int shmid;
    char *shared_memory;

    key = ftok("shmfile", 65);
    shmid = shmget(key, SHM_SIZE, 0666);
    shared_memory = (char*) shmat(shmid, NULL, 0);

    printf("Reader: Data dari shared memory: %s\n",
    shared_memory);

    shmdt(shared_memory);
    shmctl(shmid, IPC_RMID, NULL);
    return 0;
}
```

Compile:

```
gcc shm_writer.c -o shm_writer
```

```
./shm_writer
```

```
gcc shm_reader.c -o shm_reader
```

```
./shm_reader
```

Hasil:

```
aqil@DESKTOP-RGGJ3FG:~$ ./shm_writer
Writer: Masukkan data: halo aqil
Writer: Data tersimpan di shared memory.
```

```
aqil@DESKTOP-RGGJ3FG:~$ ./shm_reader
Reader: Data dari shared memory: halo aqil!
```

Analisis:

Shared Memory adalah metode komunikasi paling cepat karena proses berbagi area memori yang sama.

Namun perlu mekanisme sinkronisasi tambahan agar tidak terjadi race condition jika dua proses menulis bersamaan.

4. Percobaan 2.4 Perbandingan Performa IPC

Kode: ipc_benchmark.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/time.h>
#include <sys/ipc.h>
#include <sys/shm.h>

#define DATA_SIZE 1000000
#define SHM_SIZE DATA_SIZE * sizeof(int)

double get_time() {
```

```

    struct timeval tv;
    gettimeofday(&tv, NULL);
    return tv.tv_sec + tv.tv_usec / 1000000.0;
}

void test_pipe() {
    int pipefd[2];
    pid_t pid;
    int data[DATA_SIZE];
    double start, end;

    pipe(pipefd);
    start = get_time();
    pid = fork();

    if (pid == 0) {
        close(pipefd[1]);
        read(pipefd[0], data, sizeof(data));
        close(pipefd[0]);
        exit(0);
    } else {
        close(pipefd[0]);
        for (int i = 0; i < DATA_SIZE; i++) data[i] = i;
        write(pipefd[1], data, sizeof(data));
        close(pipefd[1]);
        wait(NULL);
        end = get_time();
        printf("Pipe: %.6f detik\n", end - start);
    }
}

```

```

    }
}

void test_shared_memory() {
    key_t key = ftok("shmfile", 65);
    int shmid;
    int *shared_data;
    pid_t pid;
    double start, end;

    shmid = shmget(key, SHM_SIZE, 0666 | IPC_CREAT);
    start = get_time();
    pid = fork();

    if (pid == 0) {
        shared_data = (int*) shmat(shmid, NULL, 0);
        int sum = 0;
        for (int i = 0; i < 100; i++) sum += shared_data[i];
        shmdt(shared_data);
        exit(0);
    } else {
        shared_data = (int*) shmat(shmid, NULL, 0);
        for (int i = 0; i < DATA_SIZE; i++) shared_data[i] = i;
        wait(NULL);
        end = get_time();
        printf("Shared Memory: %.6f detik\n", end - start);
        shmdt(shared_data);
        shmctl(shmid, IPC_RMID, NULL);
    }
}

```

```
    }  
    }  
  
    int main() {  
        printf("Benchmarking IPC Methods dengan %d  
integers\n\n", DATA_SIZE);  
        test_pipe();  
        test_shared_memory();  
        return 0;  
    }
```

Hasil:

```
aqil@DESKTOP-RGGJ3FG:~$ ./ipc_benchmark  
Benchmarking IPC Methods dengan 1000000 integers  
Pipe: 0.0008 detik  
Shared Memory: 0.0002 detik
```

Analisis:

- Pipe membutuhkan waktu lebih lama karena data harus melewati kernel buffer.
- Shared Memory jauh lebih cepat karena proses langsung membaca/menulis ke area memori yang sama.
- Ini menunjukkan efisiensi shared memory IPC untuk komunikasi data besar.

KESIMPULAN

Proses merupakan satuan kerja utama sistem operasi yang memiliki PID unik dan dapat membentuk proses anak melalui `fork()`. Komunikasi antar proses (IPC) memungkinkan pertukaran data antar proses yang berjalan bersamaan. Pipe digunakan untuk komunikasi sederhana satu arah antara parent-child, sedangkan FIFO mendukung komunikasi dua arah antar proses yang terpisah. Shared Memory menawarkan kecepatan tertinggi karena data dibagikan langsung di memori, namun membutuhkan mekanisme sinkronisasi. Performa IPC sangat bergantung pada ukuran data, efisiensi metode, dan beban sistem saat proses berjalan.